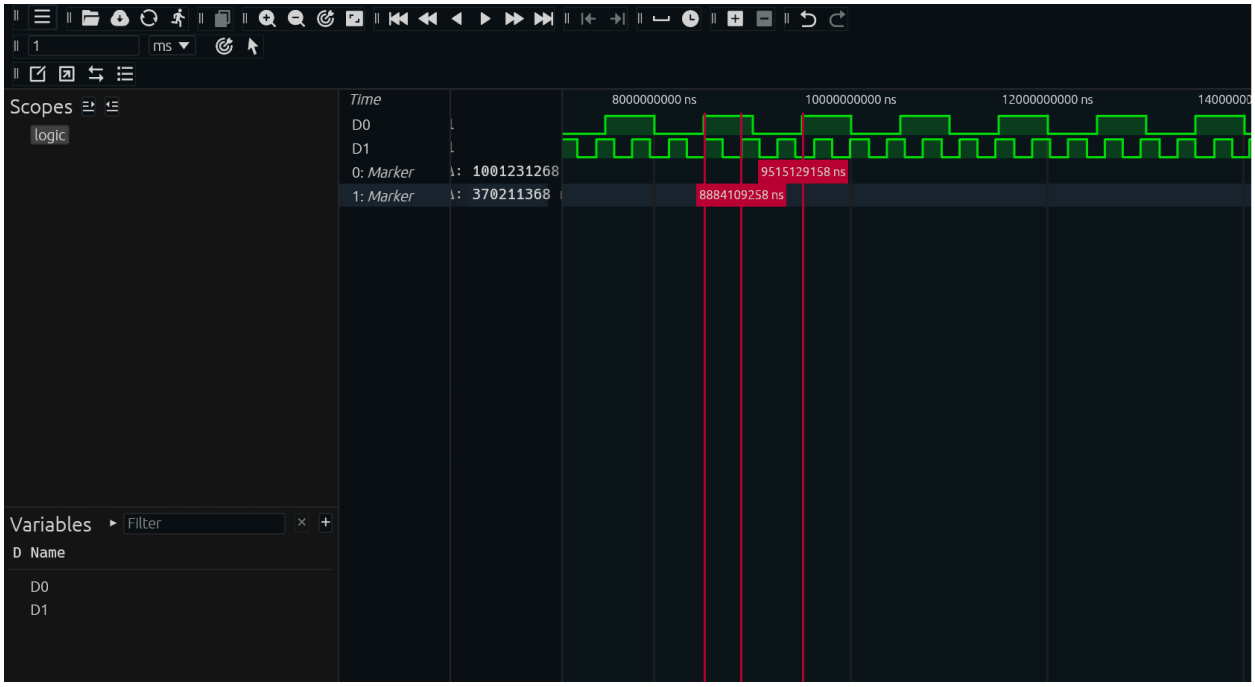
QUESTION1:

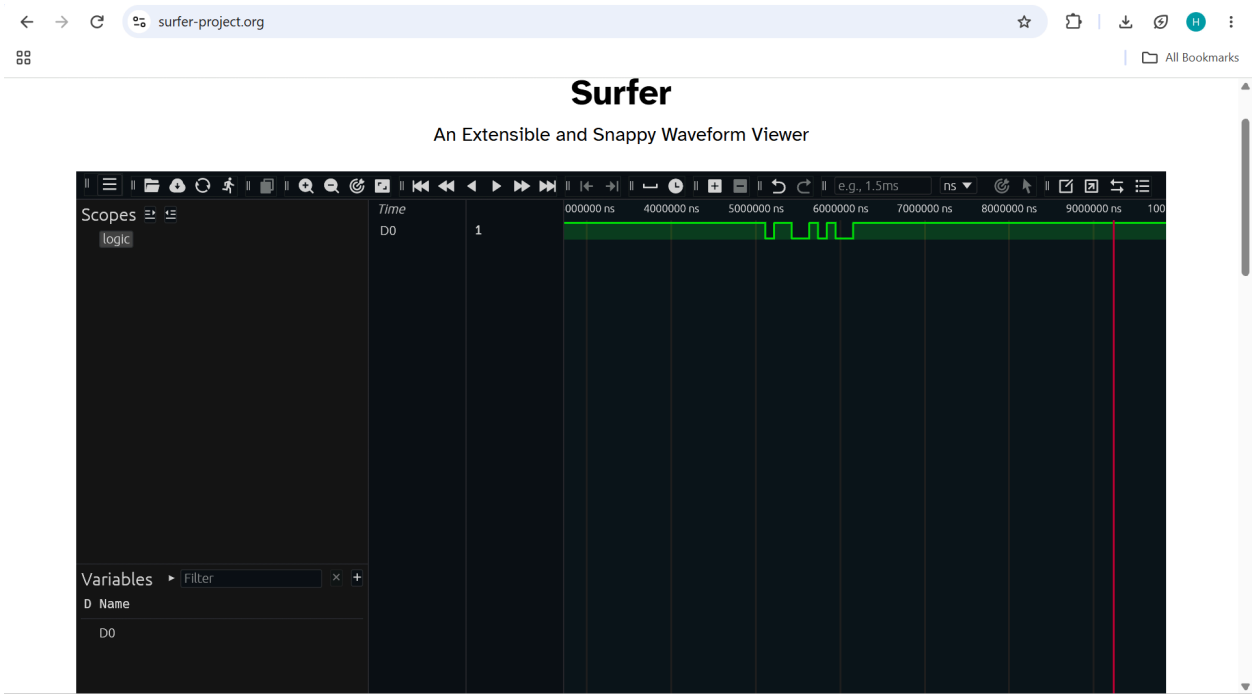
1) An Arduino essentially gives an output of 5V when its digital pins are set to HIGH. Whereas an LED, is a diode which needs around a 2V potential difference to break the barrier and allow recombination of holes and electrons across the barrier. So when an arduino is connected directly it applies a voltage of 5V across the LED, and since the diode has very little resistance a huge current flows into the diode and tries to recombine a massive no.of electrons and holes. This huge amount of recombination cannot be handled and it causes a lot of collisions which is macroscopically seen as heat. Also since a semiconductor's resistance decreases on increasing temperature, the current increases further and burns up the LED within a few seconds. The resistor limits the voltage across the LED to its barrier potential alone.
   A standard LED barrier potential=2V
   Arduino Output=5V
   Current needed=10mA
   By Ohm's Law-> 5-2/10mA=300ohms
   SIMULATION LINK:
   https://www.tinkercad.com/things/0aitrjzr791-abhiyaanq11?sharecode=UEFmuVkODIJ96a8YmBSonDbCKjUfK9LR-ZnSSZyMGpc

   2)I initially tried using delay like i did in the first one but realized delay pauses the whole code i.e for both the led's and not only the one i am writing that delay statement for. I then learned about the function millis gives the time in ms that has elapsed since the arduino started running, i then used this to measure the time elapsed since an led was previously toggled to blink the led according to the time period.
   LINK: https://wokwi.com/projects/478811847982386177



3)LINK: https://wokwi.com/projects/478818062737674241
Controlling the flickers seemed pretty hard. I initially tried controlling the noise by a distance filter like if the absolute value of (prevdist-currentdist)>=2cm but i realized that if the object inches from 0 to 400 in steps of 1cm then the servo would essentially never move. Then I thought of averaging the values but if the distances were [1,1,400] then the average wouldn't essentially move the servo as much as it should. So I switched to angle filtering. If the non-rounded value of the prevangle and currentangle differed by one then we could turn the servo, this would essentially mean the obj should move a cumulative distance of 2.2cm.

4) So the first bit is what signals the receiver line that a signal is being transmitted, the data size is 8bits, then the parity bit which makes sure the number of ones in the data+parity bit is even (8**E**1) and then the stop bit.
The data bit is essentially:01010011
Converting this binary into decimal would yield 83 which is the ASCII code for S.
The parity bit is zero which is correct as there are even number of ones in the data bits.
Since the transmission speed is 9600 baud, it means 9600 bits per sec, 11bits would take 11/9600 seconds which is roughly 0.001146 seconds.
In an Arduino transmission takes place via 8N1 protocol and not 8E1 so there is no parity bit present, but we can change it to 8E1.



The transmission at idle state stays at one and then it goes to zero to signal the start of data transmission. It then transmits the data (01010011) and then gives the parity and stop bit then goes back to idle state.
LINK for SIMULATION:https://wokwi.com/projects/478832137596861441
5) CAN, UART, I2C and SPI all are communication protocols. Each of them has their own specific advantages. Since in a car a lot of sensors and even independent systems communicate, CAN has the advantage of being able to manage simultaneously transmitted messages by prioritizing them and also swiftly communicating with every system.

UART: Its connection is more of a Peer-Peer connection. 2 Devices and 2 wires between them one for transmitting and the other for receiving. The receiving line is always HIGH when idle and when the transmission takes place a logic LOW is sent first to signal the start of data transmission. Also it is asynchronous, that is, the 2 chips agree on the speed at which bits are transferred to arrange the data accurately. {Baud Rate}

I2C: It uses a bus topology. It is synchronous and has 2 wires, one transmitting the clock signals and the other the main data. It is a master-slave network. The master calls out the unique address of one of the component systems to transmit data to it.

SPI: A direct hotline to each sensor would be a good way to describe this. It has dedicated separate lines for incoming,outgoing and clock signals, thus data transmission is fast. But it is cumbersome if the number of component systems increases.
The above three are most commonly used within one system to communicate between different parts of the system but CAN can be used to communicate with independent systems too.

CAN: It transmits the data via two twisted wires and the component systems decide if the message is for them or not.
In a CAN the protocols are structured in a way such that we can prioritize messages if they are sent simultaneously without loss of data unlike in I2C or others where simultaneous transmission results in loss of data.

The messages are sent starting with a 11 or 29 bit ID which signals the priority and the type of data being sent . In a CAN system a 0 is prioritized over a 1. So the emergency braking system would have a code like 001 and air-conditioning like 111 thus being able to prioritize the message of the emergency braking system. When a system's chip realizes it is overridden it shuts up and lets the important guys do the transmission essentially. The start of a message is signalled similar to how UART does. Also CAN has a strict protocol of not having more than 5 same logic levels in a row.

CAN also has an error management system, it can even place a specific participant on probation or even block it. It finds if a transmission has been corrupted in 4 ways:
i) A 15 bit unique code is generated by performing mathematical operations over the message and appends it to the end of the transmission, the receivers then perform the same mathematical operations and confirms it with the appended code.{ Cyclic redundancy Check}
ii) Near the end a bit is alloted one. Every receiver which has successfully performed the CRC check switches to zero, as zero is prioritized than one the main line changes to voltage low signalling that at least one receiver has heard the transmission. If the one persists then it is a signal to the transmitter that the transmission was corrupted and none of the receivers received the correct transmission and prepares to transmit once again.
iii)There are certain bits which are pre-decided to be either one or zero like the last bit is always 1, so if a zero is detected in place of a hardcoded one then the system knows something is wrong.
iv) Finally there can't be more than 5 identical bits consecutively.
When an error is discovered then any receiver transmits 6 consecutive zeroes which breaks the 4th rule and every receiver on the line dismisses the message and waits for a new transmission.

I was curious if something similar happened on our phones when we played a youtube video when music was already running. I learned that in our phones there is a boss, the OS which pretty much has similar prioritizing systems and user-hit plays are prioritized right after calls, so the OS sends a command to the music app to stop and grants youtube to use the speaker. On the other hand in CAN, if both the AC and braking systems were sending messages, they both continue to transmit messages until one of them attains priority, that is they transmit a zero instead of one. As per CAN's rules a zero is dominant and one is recessive. So let us say the AC transmitted 001111 and the braking system 001101. Only at the fifth bit will the AC transmitter realize that it sent a

one but the line is at zero and then it stops transmitting its messages. The AC then waits for the final end marker that is seven consecutive ones and then by CAN protocol 3 more ones and then starts retransmission again. But in this case the AC might just be stumped permanently by brakes, engine and all the high priority guys transmitting messages. So there is a rate limit enforced on each component. Something like an engine can send a message once per 20ms and Ac once per 500ms. Cars also do have multiple can systems to separate the high priority guys from the others. Even in the AC system if multiple messages get queued they are arranged in a priority order automatically. Also nowadays cars use CAN-FD(flexible data rate) i.e when a node wins an arbitration the speed is immediately increased 5 times to prevent bus starvation.